

Sequence Educational Game Collection

Complete Design, Content, Interface, and Engineering Specification

Reference implementation

The most recent working implementation is:

```
sequence_twenty_games_exact_position_fix.html
```

Use that file as the starting point whenever it is available. Preserve existing functionality unless a requested change explicitly replaces it.

1. Project concept

Sequence is a collection of educational logic games in which the player reconstructs a causal, procedural, scientific, technological, or historical process.

Each game contains:

- Three conceptual sequences
- Four event cards per sequence
- Twelve cards total
- A fixed starting passage before the first sequence
- Two fixed explanatory bridge passages between the sequences
- Two optional relational hints
- A strict limit of three answer checks

The player must solve two levels of order:

1. Put the four cards inside each sequence into the correct order.
2. Put the three complete sequences into the correct overall order.

The educational content must be **load-bearing**. Understanding the subject should materially help the player solve the puzzle.

The game should not reduce to sorting obvious dates or noticing words such as “first,” “next,” or “finally.”

2. Core educational philosophy

Each card should represent an event, state, transformation, observation, decision, or procedure that makes a later event possible or meaningful.

Good ordering relationships include:

- Physical causation
- Biological prerequisites
- Information dependencies
- Procedural requirements
- Instrument signal chains
- Evidence leading to decisions
- Material transformations
- Control-system responses
- Measurement and calibration pipelines
- Model-observation-update cycles

The strongest design principle is:

Each card should occur where it does because of a dependency, not merely because it happens to have an earlier date.

The opening material should teach principles that support reasoning without directly revealing the card order.

3. Audience and difficulty

Target level:

- Curious high-school students
- Undergraduates outside the field
- Military personnel learning unfamiliar technical domains
- General adult learners

Writing should be:

- Scientifically accurate
- Plain-language
- Concise enough for a game
- Substantive enough to teach real mechanisms
- Free of unnecessary jargon
- Clear about simplifications

Specialized terms are welcome when they are defined in the glossary.

4. Collection structure

The reference collection currently contains 20 games.

Original core collection

1. Medicine and biology — **The Cut That Closes**
2. Astronomy — **A Star Like the Sun**
3. Computing — **The First Visit to a Secure Website**
4. Law and civics — **A Federal Civil Case**
5. History and diplomacy — **Thirteen Days in October**
6. Aerospace engineering — **Two Stages to Orbit**
7. Chemistry — **How Soap Carries Grease Away**
8. Ecology — **From Fertilizer to a Dead Zone**
9. Earth science — **From Locked Fault to Coastal Flood**
10. Finance and payment systems — **What Happens After You Tap**

Office of Naval Research-oriented collection

1. Electro-optics and sensors — **Seeing Heat**
2. Precision navigation and estimation — **Finding Position Without GPS**
3. Oceanography and instrumentation — **Reading the Water Column**
4. Naval power and controls — **Holding the Electrical Bus**
5. Materials science and sustainment — **Stopping the Sea from Eating Steel**
6. Autonomy and multi-agent systems — **Many Robots, One Search**
7. Cybersecurity and resilient computing — **Detect, Isolate, Recover**
8. Human performance and physiology — **Heat on the Flight Deck**
9. Ocean prediction and applied mathematics — **From Ocean Sensor to Better Forecast**
10. Space weather and communications — **When the Sun Disturbs the Link**

The ONR collection is intentionally diverse and spans:

- Instruments
- Sensors
- Oceanography
- Navigation
- Power
- Materials
- Autonomy
- Cybersecurity
- Human performance
- Applied mathematics
- Space science
- Communications

At least several games should explicitly explain how instruments work.

5. Per-game content specification

Every game must contain the following fields.

```
{
  id: "unique_machine_readable_id",
  icon: "short symbol or abbreviation",
  collection: "Core collection" or "ONR collection",
  discipline: "Subject area",
  title: "Short game title",
  headline: "One-sentence player-facing challenge",
  kicker: "Scenario label",
  story: [
    "Scenario paragraph one.",
    "Scenario paragraph two."
  ],
  principles: [
    ["Principle title", "Non-answer-revealing explanation."],
    ["Principle title", "Non-answer-revealing explanation."],
    ["Principle title", "Non-answer-revealing explanation."],
    ["Principle title", "Non-answer-revealing explanation."]
  ],
  terms: [
    ["Term", "Definition"],
    ...
  ],
  note: "Simplifying assumption or caveat.",
  sources: "Short factual-verification note.",
  chapters: [
    {
      id: "chapter_one_id",
      cards: [
        ["card_id_1", "Title-only event text"],
        ["card_id_2", "Title-only event text"],
        ["card_id_3", "Title-only event text"],
        ["card_id_4", "Title-only event text"]
      ]
    },
    {
      id: "chapter_two_id",
      cards: [...]
    },
  ],
}
```

```

    {
      id: "chapter_three_id",
      cards: [...]
    }
  ],
  chapterOrder: [
    "chapter_one_id",
    "chapter_two_id",
    "chapter_three_id"
  ],
  intro: "Fixed starting-situation passage shown before Sequence A.",
  segues: [
    "Fixed bridge between the first and second sequence positions.",
    "Fixed bridge between the second and third sequence positions."
  ],
  hints: [
    "Optional relational hint connecting one card to another.",
    "Optional relational hint connecting one card to another."
  ]
}

```

6. Card-writing rules

Each game has exactly 12 cards:

- Three chapters
- Four cards per chapter
- No duplicate card IDs
- No duplicate card text within a game

Cards should contain **only the event title or event statement**.

Do not add:

- Explanatory paragraphs
- Definitions
- Dates unless genuinely necessary
- Labels such as “first,” “second,” or “final”
- Obvious sequence numbering
- The chapter’s conceptual theme

Good card:

The readout circuit measures the pixel signals

Weak card:

Next, the readout circuit measures the signals so the camera can eventually make an image

Cards should be individually understandable but should not explain the entire dependency.

7. Chapter design

The cards are pre-grouped into their correct conceptual chapters.

The player does **not** decide which four cards belong together.

The player must:

- Order the four cards within each given chapter
- Move the three entire chapter blocks into the correct global order

Each chapter should represent a meaningful intermediate process, such as:

- Acquisition
- Transformation
- Interpretation

or:

- Initiation
- Development
- Resolution

Chapter themes should not be displayed during gameplay because they could reveal the answer.

The visible chapter labels should remain generic:

- Sequence A
- Sequence B
- Sequence C

These labels refer to current screen position, not permanent chapter identity.

8. Starting-situation passage

A fixed passage must appear immediately before Sequence A.

Label:

Starting situation

Its purpose is to establish:

- The initial physical or procedural state
- What resources or information are available
- What problem the first sequence must solve
- What has not happened yet

It should guide placement of the first chapter without naming the chapter or listing its card order.

Example structure:

The instrument begins with calibrated sensors but no observations from the water column. Sequence A must place the package in the environment and ensure that several sensors characterize a common sample.

The starting passage remains fixed when chapter blocks move.

9. Bridge passages

Two fixed bridge passages appear:

- Between Sequence A and Sequence B
- Between Sequence B and Sequence C

They do not move with the chapter blocks.

Each bridge should describe:

1. The state produced by the chapter above it
2. The unresolved problem
3. What kind of transformation the next chapter must accomplish

A bridge should not list exact card titles.

Example pattern:

The instrument now has synchronized electrical observations, not direct measurements of every desired property. The next sequence must apply calibration and physical relationships to derive those properties.

The bridges are major clues for ordering the chapter blocks.

10. Opening page

The opening page should include:

- Kicker
- Headline
- Scenario
- Task description
- Four guiding principles
- Glossary
- Simplifying assumption
- Accuracy or source note
- Begin Game button

The opening page should **not** contain a prose summary of the full correct process.

Do not include an overview that effectively tells the player:

- Chapter 1 does X
- Chapter 2 does Y
- Chapter 3 does Z

Instead, provide transferable principles.

For example, an inertial-navigation briefing may explain:

- Motion sensors do not measure position directly
- Orientation controls interpretation
- Integration magnifies small errors
- Corrections should include uncertainty

These help without revealing the actual twelve-card order.

11. Guiding principles

Every game must have exactly four guiding principles.

Each principle contains:

- A short title
- A two- or three-sentence explanation

Principles should help the player reason about dependencies.

They should not repeat card text or directly state card pairs in order.

Bad principle:

The generator comes before the converter.

Good principle:

Generation and consumption must balance. A sudden mismatch changes voltage, frequency, or stored energy in the electrical bus.

Direct card-to-card relationships belong in the optional hints, not in the principles.

12. Glossary

Each game should contain approximately 5–10 useful terms.

Definitions should:

- Be one or two sentences
- Use accessible language
- Support reasoning
- Avoid revealing the full sequence

Instrument games should define important components such as:

- Transducer
 - Readout circuit
 - Detector
 - Calibration
 - Filter
 - Beamforming
 - Analog-to-digital converter
-

13. Optional hints

Each game has exactly two optional hints.

Hints are hidden initially.

Buttons:

- Reveal Hint 1
- Reveal Hint 2

After use, the button becomes disabled and says the hint was revealed.

Hints:

- Do not consume a try
- Remain visible for the rest of the playthrough
- Can be revealed independently
- State a useful relation between two cards
- Should not reveal an entire four-card sequence

Typical form:

The booster engines shut down before the empty booster separates from the upper stage.

or:

The model predicts the observed quantity before the observation-minus-forecast difference can be calculated.

Hints should usually connect two cards within the same chapter.

14. Three-check rule

The player has exactly three checks.

The interface must show:

- Checks used
- Tries remaining

A hint does not use a check.

Moving cards does not use a check.

Reshuffling unlocked items does not use a check.

Successful check

If all 12 cards and all three chapter positions are correct:

- End the game as a win
- Display the completed solution
- Show how many checks were used

A correct solution on the third check still counts as a win.

Unsuccessful first or second check

- Lock all currently correct items
- Display how many chapter positions are correct
- Display how many card positions are correct
- Show tries remaining
- Allow continued play

Unsuccessful third check

- End the game immediately
- Display a loss message
- Reveal the complete correct solution
- Offer:
 - Try a new shuffle
 - Review principles
 - Choose another game

15. Exact-position card checking

This requirement is critical.

Every card must be checked independently against its exact position.

Suppose the correct order is:

A · B · C · D

and the player submits:

B · A · C · D

Cards C and D are in their exact positions and must both:

- Turn green
- Become locked
- Be counted as correct

Do **not** check only the longest correct prefix.

The earlier prefix-based implementation contained a bug because it stopped checking after the first incorrect card.

Correct logic:

```
arr.forEach((cardId, index) => {  
  if (cardId === correct[index]) {  
    lockedCardPositions[chapterId].add(index);  
  }  
});
```

Use an independent `Set` of locked indices for each chapter:

```
lockedCardPositions = {  
  chapter_one_id: new Set(),  
  chapter_two_id: new Set(),  
  chapter_three_id: new Set()  
};
```

Do not use a single prefix count.

16. Locked-card movement behavior

A card in an exact correct position must remain fixed.

The player must still be able to rearrange unlocked cards around locked positions.

Example:

Correct:

A · B · C · D

Current state after checking:

A · C · B · D

Locked positions:

- Position 1 contains A
- Position 4 contains D

The player must be able to swap or reorder B and C without moving A or D.

Arrow behavior

Arrow buttons should move a card to the next available unlocked position in the chosen direction.

They should skip locked positions.

A locked card:

- Has disabled arrow buttons
- Is not draggable
- Appears green

Drag-and-drop behavior

Dragging an unlocked card must reorder only the sequence of unlocked cards.

Locked cards must remain in their exact indices.

Do not implement drag-and-drop with a simple array splice if that would shift locked cards.

Recommended strategy:

1. Determine all unlocked indices
2. Extract the cards currently in those indices
3. Reorder only that extracted list
4. Write it back into the same unlocked indices

Reshuffling behavior

“Reshuffle unlocked items” must:

- Preserve locked chapter positions
- Preserve every locked card position
- Shuffle only cards in unlocked indices
- Preserve attempts used
- Preserve revealed hints

A full reset or “Play again” may reset everything.

17. Chapter-position checking

Each chapter has a permanent internal ID.

The visible labels Sequence A, B, and C refer only to screen position.

When checking:

```
chapterOrder.forEach((chapterId, index) => {  
  if (chapterId === current.chapterOrder[index]) {  
    lockedChapters.add(chapterId);  
  }  
});
```

A correctly placed chapter:

- Turns green
- Cannot be dragged
- Cannot be moved with arrows
- Remains fixed during reshuffling

The cards inside a correctly positioned chapter may still be reordered unless those cards are individually locked.

18. Gameplay controls

Each card should support:

- Drag-and-drop
- Up arrow
- Down arrow

Each chapter should support:

- Drag-and-drop using a chapter handle
- Up arrow
- Down arrow

Other buttons:

- Check both levels
- Reshuffle unlocked items
- Review principles
- Reveal Hint 1
- Reveal Hint 2
- All games

After completion:

- Play again or Try a new shuffle
- Review principles
- Choose another game

19. Desktop layout

On desktop, all four cards in a chapter must fit on one horizontal line.

Required CSS concept:

```
.cards {  
  display: grid;  
  grid-template-columns: repeat(4, minmax(0, 1fr));  
  gap: 8px;  
}
```

Cards should be compact but readable.

Suggested card structure:

- Position number
- Drag handle
- Card title
- Movement buttons on a second row

Avoid very tall cards.

20. Responsive layout

Recommended behavior:

Wide desktop

- Four cards per row
- Two-column game-selection grid
- Two-column principles and glossary grids

Medium screens

- Two cards per row
- One-column home and glossary sections if needed

Small screens

- One card per row
- Chapter controls wrap below title
- Intro and bridge margins shrink
- All buttons remain usable by touch

The desktop four-card row is the preferred primary presentation.

21. Visual design

Current aesthetic:

- Dark navy background
- Layered blue panels
- Gold accents
- Cyan informational accents
- Green correct states
- Red failure or incorrect states
- Rounded cards and panels
- High contrast
- Clear but restrained shadows

Important colors should be defined as CSS variables.

Example categories:

```
--bg  
--panel  
--panel2  
--line  
--text  
--muted
```



```
--blue
--gold
--green
--red
--orange
```

Use green only for confirmed correct items or success.

Use red for failed checks or loss states.

The introductory passage should use a blue accent.

Bridge passages should use a gold accent.

22. Home screen

The home screen should show all games as selectable cards.

Each home card includes:

- Icon
- Collection label
- Discipline
- Title
- Headline
- Open Game indicator

The home screen should explain:

- Three chapters
- Four events each
- Three tries
- Two optional hints
- Correct positions lock
- The third unsuccessful check reveals the solution

The current collection labels are:

- Core collection
 - ONR collection
-

23. Solution display

Both the win and loss screens should display:

1. Starting situation
2. Correct Chapter 1 with its four cards
3. First transition
4. Correct Chapter 2 with its four cards
5. Second transition
6. Correct Chapter 3 with its four cards

The solution should use the same card titles.

Do not introduce new explanations for individual cards unless specifically requested.

The solution view may label the groups:

- Chapter 1
 - Chapter 2
 - Chapter 3
-

24. Simplification and factual accuracy

Each game must contain a caveat explaining where reality is more complicated.

Examples:

- Biological phases overlap
- Browser operations can occur in parallel
- Rocket event timing varies by vehicle
- Real power systems use several architectures
- Data-assimilation systems use complex covariance models
- Instrument designs vary by detector or frequency band

Do not allow a simplified linear game to imply that every real system is strictly sequential.

The source note should name the kinds of authoritative references used, such as:

- NASA
- NOAA
- NIST
- NIH/NCBI
- NIST cybersecurity guidance
- U.S. Courts

- Federal Reserve
- Office of Naval Research program descriptions
- Major museums or educational institutions

Raw URLs are not required inside the game.

25. Topic-selection criteria for future games

A proposed topic should satisfy most of the following:

1. It represents a meaningful field not already overrepresented.
2. It has a real dependency chain.
3. It supports exactly three meaningful conceptual chapters.
4. Each chapter supports four distinguishable events.
5. The player can reason from principles rather than memorize dates.
6. Two relational hints can help without solving the game.
7. The process is useful or interesting to the target audience.
8. It can be explained accurately at approximately a high-school level.
9. The beginning and end are understandable, while the middle contains educational value.
10. The simplification can be clearly disclosed.

Avoid topics where:

- Most events occur simultaneously
 - Several orders are equally valid
 - The sequence relies entirely on arbitrary dates
 - The card wording itself gives away the answer
 - The process cannot be represented without major falsehoods
 - All three chapters would teach essentially the same mechanism
-

26. Instrument-game requirements

For “how an instrument works” games, try to cover three levels:

Chapter 1: Physical interaction or acquisition

Examples:

- Radiation enters an optical system
- Sound reaches a transducer
- Water passes through a sensor
- Motion affects an inertial sensor

Chapter 2: Electrical or computational conversion

Examples:

- Detector response
- Readout electronics
- Amplification
- Filtering
- Frequency conversion
- Digitization

Chapter 3: Calibration, interpretation, or output

Examples:

- Correcting sensor offsets
- Combining channels
- Deriving physical quantities
- Estimating uncertainty
- Producing an image or track
- Entering a model

Instrument games should explain both physics and electronics rather than treating the device as a black box.

27. Code architecture

The implementation should remain a standalone offline HTML file containing:

- HTML
- CSS
- JavaScript
- Embedded game data

No server is required.

No external libraries should be required.

Primary state variables should include:

```
let current = null;
let chapterOrder = [];
let cardOrders = {};
let lockedChapters = new Set();
```

```
let lockedCardPositions = {};  
let attempts = 0;  
let hintsUsed = [false, false];  
let feedback = "";  
let feedbackKind = "";
```

Important utility functions include:

```
shuffle()  
chapterById()  
cardTitle()  
correctCards()  
resetGame()  
goHome()  
chooseGame()  
renderHome()  
renderBrief()  
renderPlay()  
renderChapter()  
checkAnswer()  
renderSolved()  
renderLost()  
solutionMarkup()  
revealHint()  
reshuffleUnlocked()  
isCardLocked()  
nextUnlockedIndex()  
reorderUnlockedCards()
```

28. Required validation

Before delivering a revised collection, validate all of the following.

Data validation

- Correct number of games
- Unique game IDs
- Every game has exactly three chapters
- Every chapter has exactly four cards
- Every game has exactly 12 unique card IDs
- Every game has 12 unique card texts
- Every game has exactly four principles
- Every game has exactly two hints

- Every game has exactly two bridges
- Every game has one starting passage
- `chapterOrder` references the three valid chapter IDs

JavaScript validation

Extract the script and run:

```
node --check file.js
```

The syntax check must pass.

Exact-position property testing

Test all 24 possible permutations of a four-card chapter.

For every permutation:

- Determine which indices contain the exact correct card
- Run the game's checking logic
- Verify that precisely those indices lock

Across 20 games and 60 chapters, this means:

- 60 chapters × 24 permutations
- 1,440 permutation tests

Movement testing

Test that:

- Arrow movement skips locked positions
- Drag reordering preserves locked indices
- Separated locked positions remain stable
- Repeated unlocked reshuffles never move locked cards
- Locked chapters never move
- Checks do not reset when reshuffling
- Hints remain revealed after movement or reshuffling

Attempt testing

Verify:

- First incorrect check leaves two tries
- Second incorrect check leaves one try

- Third correct check wins
 - Third incorrect check loses
 - Loss displays the complete correct solution
 - Hints do not consume tries
-

29. Delivery requirements

When producing or updating the collection:

1. Deliver one standalone `.html` file.
2. Preserve all existing games unless asked to remove or replace them.
3. Name the file descriptively.
4. Provide a direct download/open link.
5. State what changed.
6. State the validation performed.
7. Be honest about anything not tested.

The current preferred reference filename is:

```
sequence_twenty_games_exact_position_fix.html
```

30. Future-request prompt

Use the following request when returning this specification:

Build or revise the Sequence educational game collection according to the attached full specification. Start from the attached latest HTML implementation when available. Preserve all existing content and mechanics unless I explicitly request a replacement. Maintain three four-card chapters per game, a fixed starting passage, two bridges, four non-answer-revealing principles, two optional relational hints, a strict three-check limit, exact-position card locking, title-only cards, responsive four-across desktop layout, and complete solution reveal on loss. Run the full syntax and behavioral validation before delivering the revised standalone HTML file.